

堆场大中小计划算法说明文档

flat_full_yard_plan_1.4_scip

目录

1	文档范围	2
2	总体算法流程	2
2.1	三层计划的数据传递关系	3
3	输入数据口径	3
3.1	统一输入对象	3
3.2	流向和箱区功能	3
3.3	用户指定箱区和贝位	4
3.4	在场箱和中转箱	4
4	大计划算法	5
4.1	大计划粒度	5
4.2	大计划输入	5
4.3	决策变量	5
4.4	主要约束	6
4.5	目标函数	6
4.6	大计划输出	7
5	中小计划算法	7
5.1	ProblemData	7
5.2	属性分组规则	8
5.3	实际需求计算	9
5.4	大计划在中小计划中的作用	9
5.5	候选列	10

目录	2
5.6 中小计划目标函数和位置选择	10
5.6.1 硬约束过滤	10
5.6.2 候选列成本	11
5.6.3 主问题惩罚	11
5.7 主问题	12
5.8 Repair 阶段	13
5.9 单独小计划模式	13
5.10 箱区内重排	13
6 输出文件	14
7 可调参数	14
7.1 YardPlanAlgorithmConfig	15
7.2 LargePlanningConfig	15
7.3 大计划目标权重 YardPlanningWeights	15
7.4 ColumnGenerationConfig: 列生成规模和早停	16
7.5 ColumnGenerationConfig: 候选贝位和初始列	16
7.6 ColumnGenerationConfig: 时间预算和 SCIP	17
7.7 ColumnGenerationConfig: Primal expansion 和 Repair	17
7.8 ColumnGenerationConfig: 需求和配额模式	18
7.9 ColumnGenerationConfig: 未放置和用户要求	18
7.10 ColumnGenerationConfig: 中计划集中度	18
7.11 ColumnGenerationConfig: 大计划继承	19
7.12 ColumnGenerationConfig: E 开头出口箱区规则	19
7.13 ColumnGenerationConfig: 小计划拆分、堆场现状和作业偏好	19
8 常用运行命令	20
8.1 跑全流程	20
8.2 单独跑中小计划	20
8.3 单独跑小计划	20

1 文档范围

本文档说明 `flat_full_yard_plan_1.4_scip` 的堆场计划算法，包括大计划、中计划、小计划的输入口径、约束逻辑、目标函数、输出文件，以及主要可调参数。

本文档对应的主要代码文件如下：

- 全流程入口： `run_flat_full_yard_plan.py`
- 单独中小计划入口： `run_medium_small_column_generation.py`
- 单独小计划入口： `run_small_plan_from_medium.py`
- API 入口和统一配置： `api/planning_api.py`
- 大计划输入构建： `adapters/input_adapter_standard.py`
- 大计划求解器： `large_plan/solver.py`
- 中小计划列生成求解器： `medium_small/column_generation_planner.py`
- 中小计划问题数据结构： `block_bay_planning/models.py`

2 总体算法流程

算法采用三层计划：

1. **大计划**：按航次、流向、箱区、20/40 尺口径分配计划箱量。
2. **中计划**：在大计划箱区结果基础上，按用户输入的粗属性组规则分配到箱区和贝位。
3. **小计划**：按用户输入的细属性组规则，以及贝位、排、重量等细分规则，分配到具体贝位、块和行。

全流程入口 `run_flat_full_yard_plan.py` 的主要步骤为：

1. 读取 `InputAdapterGd` JSON，默认路径为 `data/input/input_data.json`。
2. 构建大计划输入，包括航次、箱区功能、容量、距离、在场箱快照、历史大计划、用户箱区指令、TOPS 和关闭贝位等。
3. 调用大计划 MIP 求解器，生成 `allocation.csv`。
4. 将大计划结果以内存方式传入中小计划。
5. 构建中小计划 `ProblemData`，包括实际需求、贝位容量、箱区功能、用户箱区和贝位控制、大计划配额、堆场现状协调信息等。
6. 调用中小计划列生成求解器，生成中计划、小计划、未放置箱和诊断信息输出。

2.1 三层计划的数据传递关系

层级	输入重点	输出及下游作用
大计划	航次需求、箱区功能、箱区容量、在场快照、历史计划、用户箱区规则	输出航次、流向、箱区、尺寸口径的计划量；中小计划将其作为箱区继承目标和配额引导
中计划	实际需求、大计划箱区配额、贝位容量、箱区功能、用户粗属性规则、用户箱区规则、堆场现状协调信息	输出粗属性组到箱区和贝位的分配结果；小计划继承其箱区和贝位边界
小计划	资料箱、用户细属性规则、贝位和行容量、属性混放规则、中计划边界、堆场现状协调信息	输出细属性组的具体贝位、块和行分配，并记录未放置箱

3 输入数据口径

3.1 统一输入对象

主输入为 InputAdapterGd JSON。全流程默认读取：

```
flat_full_yard_plan_1.4_scip/data/input/input_data.json
```

该 JSON 内包含航次、在场箱、资料箱、预测箱、箱区功能、贝位容量、用户规则等结构化数据。全流程直接基于该对象构建大计划和中小计划输入。

3.2 流向和箱区功能

中小计划使用的流向归并规则如下：

原始状态	中小计划流向
OF	OF
IF	IF
IZ	IZ
T	T
其他	OZ

箱区是否可放某流向箱，由箱区功能表决定。一个箱区可以标注多个功能，表示该箱区可以服务多个流向。

中小计划候选判断包含大计划继承规则：如果某箱区已经是该组的大计划分配箱区，则候选判断会认为该箱区支持该组。在构建 `ProblemData` 时，大计划行会先经过用户范围、关闭箱区和箱区功能过滤，因此功能不匹配的大计划行通常不会进入有效大计划配额。

3.3 用户指定箱区和贝位

用户箱区控制会进入大计划和中小计划输入构建：

- 允许箱区：限制某航次可用箱区范围。
- 增加箱区：作为 `required` 或 `priority` 类目标，尽量使用。
- 禁用或移除箱区：从允许范围中剔除。
- 优先箱区：在候选裁剪时优先保留。

中小计划中，用户指定箱区不会突破箱区功能。有效可用箱区可以理解为：

有效箱区 = 用户允许范围 $\cap$ 箱区功能匹配 $\cap$ 未关闭箱区 $\cap$ 有容量箱区

小计划还支持用户贝位控制：

- `required bay`：相关组优先或必须使用指定贝位。
- `blocked bay`：相关组不能使用指定贝位。

3.4 在场箱和中转箱

中小计划在两处使用在场箱：

1. 计算实际待分配需求时，扣除目标航次已经在场的箱。
2. 构建堆场现状协调信息，用于在位置选择时参考目标航次已有在场箱的分布。

同时关联进口航次和出口航次的在场中转箱会被排除。判断口径为：

`IYC_EVOY_ID` 非空 $\wedge$ `IYC_IVOY_ID` 非空

这类箱不会参与中小计划需求扣减，也不会进入堆场现状协调信息。

4 大计划算法

4.1 大计划粒度

大计划按以下维度分配：

航次 v , 流向 f , 箱区 a , 尺寸口径 $s \in \{20, 40\}$

45 尺在大计划中归入 40 尺口径；中小计划落贝时仍保留 45 尺自身约束。

4.2 大计划输入

大计划输入由 `build_large_inputs()` 构建，核心数据包括：

- 航次集合 V
- 流向集合 F
- 箱区集合 A
- 需求 D20、D40
- 箱区容量 C20、C40
- 箱区作业能力上限 H
- 航次到箱区距离 `distance`
- 箱区功能 U
- 箱区可用性 E20、E40
- 当前快照箱量 $S/L/Q$
- 历史计划 P
- TOPS 保留和关闭影响
- 用户箱区控制
- 泊位冲突航次对

4.3 决策变量

变量

含义

$x_{20}[v, f, a]$	航次、流向、箱区的 20 尺计划量
$x_{40}[v, f, a]$	航次、流向、箱区的 40 尺计划量
$y[v, a]$	航次是否使用箱区
s_{20}/s_{40}	未满足需求松弛量
h	箱区服务航次数超额
o	作业能力超额
m_{20}/m_{40}	尺寸分布均衡变量
r_{pos}/r_{neg}	相对历史计划的调整幅度
$required_area_unmet$	用户要求箱区未满足松弛量

4.4 主要约束

- 需求满足：计划量加未满足量覆盖需求。
- 箱区容量：箱区内 20/40 箱量不能超过可用容量。
- 箱区功能：箱区功能表决定可服务流向。
- 箱区可用性：关闭、TOPS、功能、容量等影响 E20/E40。
- 作业能力：新增箱量受 H 和软超额变量控制。
- 航次使用箱区数量：通过 y 和相关惩罚控制。
- 出口 OF 作业路数：限制出口航次使用 OF 箱区数量。
- 泊位冲突：预计靠泊时间接近的出口航次共用箱区会被惩罚。
- 历史计划延续：对旧计划调整量施加惩罚。
- 用户指定箱区：允许范围作为硬约束，要求箱区通过高权重软惩罚尽量满足。

大计划不落到具体贝位，因此不检查贝位层面的属性混放、行容量、同粗属性聚集等细粒度规则。

4.5 目标函数

大计划使用加权和目标，主要包含：

- 未满足需求惩罚。
- 作业能力超额惩罚。
- 出口 OF 箱区使用惩罚。
- 距离成本。

- 箱区服务航次数超额惩罚。
- 泊位冲突惩罚。
- 相对历史计划调整惩罚。
- 尺寸分布均衡惩罚。
- 用户要求箱区未满足惩罚。

4.6 大计划输出

大计划输出到：

`outputs_large/latest_run/allocation.csv`

常见字段包括 `voy_id`、`flow`、`area_no`、`size`、`planned_qty`、`new_qty`。

`planned_qty` 表示规划后该航次、流向、箱区、尺寸的总计划量；`new_qty` 表示仍需新增安排的箱量。

5 中小计划算法

5.1 ProblemData

中小计划核心输入为 `ProblemData`：

- `groups`：中计划粗属性组。
- `small_groups`：小计划细属性组。
- `bays`：贝位容量和属性状态。
- `big_plan`：有效大计划行。
- `assigned_areas`：大计划已分配箱区。
- `area_quota`、`area_size_quota`：大计划箱区和尺寸配额。
- `area_functions`：箱区功能表。
- `existing_coarse_area_load`、`existing_coarse_bay_load`：堆场现状协调信息。
- `user_voyage_area_allowlist/blocklist/requirements`：用户箱区控制。
- `user_group_bay_requirements/blocklist`：用户贝位控制。
- `attribute_rules`：由输入 JSON 读取并规范化后的粗属性、细属性、贝位混放、行混放、重量档规则。

5.2 属性分组规则

中计划和小计划的属性组不是固定写死的，而是由输入 JSON 中的规则字段决定。代码先将这些字段读入 `AttributeRules`，再按航次取对应规则：

输入字段	作用位置	规则含义
<code>rough_attr</code>	中计划粗属性组	决定出口箱中计划需求如何拆成粗属性组。
<code>detail_attr</code>	小计划细属性组	决定资料箱如何在粗属性组基础上进一步拆成细属性组。
<code>bay_rules</code>	小计划贝位规则	指定同一贝位不能混放的属性。该字段会并入小计划分组和贝位兼容性判断。
<code>row_rules</code>	小计划排规则	指定同一排不能混放的属性。该字段会并入小计划分组和排兼容性判断。
<code>weight_level</code>	小计划重量档	为航次配置重量分档；配置后小计划会将 <code>IYC_CWEIGHT</code> 按重量档纳入分组。

这些字段既可以作为一套全局规则提供，也可以按航次分别提供。这里的“读取”指需求分组和属性值生成阶段：中计划按当前航次取得 `rough_attr` 作为粗属性字段；如果 `rough_attr` 中包含 `IYC_CWEIGHT`，则该字段会按当前航次的 `weight_level` 转成重量档。小计划按“粗属性字段 + 当前航次的 `detail_attr`”生成细属性组，并额外携带 `bay_rules`、`row_rules` 和重量档信息，供后续贝位、排兼容性和重量分组使用。

求解阶段不是只检查粗属性或细属性本身。中计划和小计划共用候选列生成与 `repair` 逻辑，候选位置会按当前航次读取 `bay_rules` 和 `row_rules`，检查在场箱、已选新箱、贝位容量、行容量、尺寸和箱区功能等硬约束。

出口箱的粗属性组由当前航次的 `rough_attr` 决定，细属性组由“粗属性组 + 当前航次的 `detail_attr`”决定。

进口或其他非出口箱使用单独口径：粗属性组固定为尺寸 + 二程船 `IYC_EVOY_ID`；如果没有二程船，则使用尺寸 + 卸货港。细属性组在该粗属性组基础上叠加当前航次 `detail_attr` 中配置的字段。

中计划组的生成逻辑为：先计算航次、流向、尺寸、港口层面的实际需求，再按上述粗属性口径拆成若干粗属性组。如果资料箱中存在对应记录，则按资料箱中的真实属性比例分摊需求；如果没有资料箱记录，则使用默认或兜底属性值。

小计划组的生成逻辑为：以资料箱为基础，先扣除已经在场的箱，再按上述细属性口径形成细属性组。`bay_rules`、`row_rules` 和重量档字段会随小计划组携带，用于贝位、排和重量相关约束，但不额外定义粗属性组。堆场现状协调信息也使用同一粗属性口径，不再使用代码内部固定的粗属性组合。

5.3 实际需求计算

中小计划需求由 `calculate_medium_demands()` 计算。需求口径为资料箱、预测箱和在场箱扣减后的实际需求，大计划 `new_qty` 不作为需求上界。

需求来源包括：

- 资料箱。
- 预测箱。
- 在场箱扣减。
- 航次收箱窗口比例。

在场箱扣减会按箱号或箱 ID 去重，并排除同时关联进口航次和出口航次的在场中转箱。该排除口径也用于堆场现状协调信息，避免此类中转箱同时影响进口和出口计划。

收箱阶段比例：

阶段	比例
未开港且未来一个 horizon 内开港	0.70
未开港且未来一个 horizon 内不开港	0.00
开港后第一个 horizon	0.70
开港后第二个 horizon	0.90
开港后第三个 horizon 或更晚	1.00

默认 `horizon_hours` 为 24。每个航次按收箱开始时间和当前规划时间确定当前窗口。

5.4 大计划在中小计划中的作用

大计划在中小计划中主要承担箱区继承和配额引导作用：

1. 提供箱区继承目标。
2. 提供候选范围优先级。
3. 提供箱区和尺寸配额，用于目标函数中的偏离惩罚。
4. 在 `repair` 的前几个阶段中控制候选范围。

因此，即使实际需求与大计划配额不一致，中小计划仍按实际需求尝试落贝，并通过软惩罚尽量继承大计划。

5.5 候选列

中小计划列生成中的一个 column 表示：

某个细属性组在某个贝位或块内放置若干箱

column 包含航次、流向、港口、尺寸、箱高、重量档、特殊属性、箱区、贝位、块、放置数量、行分配、大计划配额 key、粗属性 key 和内生成本。

其中，细属性组来自 `small_groups`，其属性字段由输入 JSON 中的 `detail_attr` 及进口箱特例决定；粗属性 key 使用同一属性分组规则，是规划器在中小计划内部用于聚合、集中度和堆场现状协调的 key。

生成 column 时会检查：

- 贝位容量和行容量。
- 箱区功能。
- 用户箱区和贝位控制。
- 大计划区域优先和 fallback。
- 当前在场箱属性混放。
- 堆场现状协调成本。
- E 开头出口箱区的额外规则。

5.6 中小计划目标函数和位置选择

中小计划的位置选择不是单一排序规则，而是“先过滤、再计价、最后整体选择”的过程。候选位置必须先满足硬约束；通过硬约束后，每个候选列会得到一个列成本；主问题再在全部候选列之间选择组合，使总成本最低。

5.6.1 硬约束过滤

硬约束决定某个箱区、贝位、排是否允许使用。主要过滤条件包括：

- 箱区功能必须匹配流向；出口箱使用 OF 功能箱区，进口箱按 IZ、IF、T 或 OZ 匹配功能表。
- 用户禁用箱区、限制箱区、关闭箱区、禁用贝位和 TOPS 保留位置会作为硬过滤。
- 贝位必须支持对应尺寸；20 尺、40 尺、45 尺会按贝位能力、大贝 partner 和边贝规则生成可行位置。

- 贝位容量、行容量、尺寸容量和物理容量不能超限。
- 当前在场箱和已选候选列形成的贝位、排属性状态必须满足 `bay_rules` 和 `row_rules`。
- 单独小计划入口中，如果配置了外部中计划配额，`repair` 的部分阶段会把中计划箱区或贝位配额作为阶段性范围约束。

用户指定箱区不会强行覆盖箱区功能。也就是说，可用箱区是用户范围、关闭/禁用规则、箱区功能、容量和属性规则共同过滤后的交集。

5.6.2 候选列成本

候选列成本描述“把某个细属性组的一部分箱放到某个贝位或块”本身是否合适。主要成本项如下：

成本来源	位置选择含义
大计划继承	大计划精确配额箱区成本最低；同组大计划箱区、任意大计划箱区、非大计划箱区按 <code>fallback</code> 层级逐步加惩罚。
用户偏好	用户要求或优先箱区、贝位会降低候选成本；用户禁用范围已经在硬约束中过滤。
堆场现状	候选贝若已有同内部粗 <code>key</code> 的在场箱会得到奖励，邻近已有同 <code>key</code> 贝位也会得到较小奖励；与其他 <code>key</code> 冲突或邻近冲突会增加成本。
作业位置	航次泊位到箱区的距离、当前作业箱区、作业窗口后的箱区偏好会进入候选成本。
贝位形态	<code>fallback</code> 贝位、非优先 6 贝块等会增加成本。
E 区出口规则	E 开头箱区仍按功能表判断；对出口航次还会控制同一箱区使用的贝位起点数，并对非 40/45 尺出口箱增加软惩罚。

5.6.3 主问题惩罚

主问题不是逐列贪心选择，而是在全部候选列之间求一个组合。其目标可以概括为：

$$\begin{aligned} \min \quad & \text{候选列成本} + \text{未放置惩罚} + \text{大计划偏离惩罚} \\ & + \text{中计划集中度惩罚} + \text{小计划拆分惩罚} + \text{其他作业偏好成本} \end{aligned}$$

其中最关键的组合层目标如下：

目标类别	组合选择含义
减少未放置箱	未放置箱有高惩罚，主问题优先覆盖实际需求。

大计划偏离	按航次、流向、箱区、尺寸统计实际使用量，与大计划配额目标的差值越大惩罚越高。
中计划集中度	对需要集中的内部粗 key，跨多个箱区会产生惩罚，最大箱区份额会产生奖励；对较大的组，会控制打开箱区数量、过小箱区份额和箱区间分布失衡。
小计划拆分	同一细属性组跨箱区、跨块、跨贝都会增加惩罚；同一内部粗 key 在同箱区内跨块、跨贝也会增加惩罚。
属性兼容性	主问题会继续保证同一贝位或同一排内不会选择互相冲突的属性取值。

因此，中小计划最终输出的位置，是在满足功能、容量、用户范围和属性混放等硬约束的基础上，对大计划继承、实际需求覆盖、粗细属性集中、堆场现状、作业距离和特殊箱区规则加权后的整体最优或近似最优组合。

5.7 主问题

列生成主问题选择 column 覆盖所有细属性组需求：

$\min \quad \text{column cost} + \text{unplaced penalty} + \text{split penalty} + \text{big plan deviation} + \text{other penalties}$

约束包括：

- 每个 group 的选择箱量加未放置量等于需求。
- 贝位容量不超限。
- 行容量不超限。
- 大计划配额根据阶段作为软目标或硬约束。

求解策略为：

1. 生成初始列。
2. 求 restricted master LP。
3. pricing 生成新列。
4. 按改善幅度、迭代次数、时间预算早停。
5. primal expansion 扩展可行列。
6. repair 尝试修复未放置箱。

7. MIP 或 fallback 得到整数解。

如果 SCIP 不可用或失败，会进入 greedy fallback。

5.8 Repair 阶段

Repair 阶段如下：

阶段	范围	大计划配额	中计划配额
stage1a	严格大计划配额范围	强制	强制
stage1b	同组大计划箱区	不强制	强制
stage2	任意大计划箱区	不强制	强制
stage3	功能匹配的堆场箱区	不强制	不强制

Repair 阶段用于在列生成主问题之后补放未放置箱。其基本策略是先更贴近大计划和中计划边界的范围内尝试放置；如果仍有未放置箱，再逐步放宽候选范围，在可用堆场范围内寻找可行位置。

表中的“中计划配额”主要针对单独小计划入口。该入口读取外部 `medium_plan.csv` 时，会将外部中计划转换为 `medium_plan_quota` 和 `medium_plan_bay_quota`，用于约束小计划 repair 是否必须留在既定中计划箱区和贝位边界内。

5.9 单独小计划模式

单独小计划入口读取外部 `medium_plan.csv`。其配置函数会设置：

- `demand_mode="doc-only"`
- `medium_plan_quota`：外部中计划箱区配额
- `medium_plan_bay_quota`：外部中计划贝位配额
- `repair_can_exceed_medium_plan_quota=True`

5.10 箱区内重排

repair 后存在箱区内重排逻辑：

- 保持箱区层面的箱量不变。
- 尝试重新选择箱区内贝位 `column`。
- 改善小计划集中度、贝位和块拆分、堆场现状协调等指标。

- 如果重排后评分没有改善，则保留原方案。

该逻辑由 `post_repair_area_relayout_enabled` 控制。

6 输出文件

全流程输出目录为：

`full_plan_outputs/<run_name>/`

文件	内容
<code>outputs_large/latest_run/allocations.csv</code>	大计划分配结果
<code>outputs_large/state/</code>	大计划历史状态，未禁用时写入
<code>medium_small_plan/medium_demand_by_port.csv</code>	中计划实际需求明细
<code>medium_small_plan/medium_plan.csv</code>	中计划结果
<code>medium_small_plan/small_plan.csv</code>	小计划结果
<code>medium_small_plan/small_plan_six_block_summary_blocks.csv</code>	小计划按 6 贝块汇总
<code>medium_small_plan/unplaced_boxes.csv</code>	未放置箱明细
<code>medium_small_plan/big_plan_used.csv</code>	中小计划使用的大计划行
<code>medium_small_plan/generated_columns.csv</code>	生成列明细
<code>medium_small_plan/diagnostics.json</code>	中小计划诊断信息
<code>pipeline_summary.json</code>	全流程摘要

7 可调参数

可调参数分为三层：

1. `YardPlanAlgorithmConfig`：全流程和 API 级参数。
2. `LargePlanningConfig` 与 `YardPlanningWeights`：大计划参数。
3. `ColumnGenerationConfig`：中小计划列生成参数。

7.1 YardPlanAlgorithmConfig

定义位置: `api/planning_api.py`

参数	默认值	说明
<code>large</code>	默认大计划配置	大计划配置对象
<code>medium_small</code>	默认中小计划配置	中小计划配置对象
<code>planning_time</code>	None	规划时间; 为空时使用输入 JSON 的规划时间
<code>medium_voyages</code>	None	指定中小计划航次; 为空时从大计划推断
<code>horizon_hours</code>	24.0	中小计划收箱窗口长度
<code>misplaced_bay_exclusion_ratio</code>	2/3	识别错放或异常贝位的阈值比例
<code>large_time_limit</code>	120.0	大计划求解时间限制, 秒
<code>large_mip_gap</code>	0.001	大计划 MIP gap
<code>large_verbose</code>	True	是否打印大计划日志
<code>disable_default_flow_aliases</code>	False	是否禁用默认 flow alias

7.2 LargePlanningConfig

参数	默认值	说明
<code>flow_aliases</code>	IE/RF/RE → OZ	大计划默认流向归并
<code>berth_conflict_threshold_hours</code>	2.0	泊位冲突航次的时间阈值
<code>weights</code>	默认权重	大计划目标函数权重
<code>required_area_penalty</code>	10000.0	用户要求箱区未满足惩罚
<code>allow_unmet_demand</code>	True	是否允许未满足需求松弛
<code>strict_validation</code>	True	是否严格校验模型参数索引

7.3 大计划目标权重 YardPlanningWeights

参数	默认值	说明
<code>miss</code>	100.0	未满足需求惩罚
<code>operation</code>	50.0	作业能力超额惩罚

of_area	40.0	出口 OF 箱区使用数量惩罚
distance	30.0	航次到箱区距离成本
share	20.0	箱区服务航次数超额惩罚
berth_conflict	25.0	泊位冲突航次共用箱区惩罚
adjustment	10.0	相对历史计划调整惩罚
balance	1.0	尺寸分布均衡惩罚
priority_area	0.01	优先箱区弱偏好

7.4 ColumnGenerationConfig: 列生成规模和早停

参数	默认值	说明
max_iterations	30	pricing 最大迭代次数
columns_per_iteration	2500	每轮最多新增列数
stalled_pricing_columns	500	停滞时的列数量阈值
min_pricing_iterations	3	至少执行的 pricing 轮数
pricing_early_stop_new_columns	0	新列数量早停阈值
pricing_max_no_improve_iterations	3	连续无明显改善轮数上限
pricing_min_lp_improvement	10000.0	LP 目标绝对改善阈值
pricing_min_lp_improvement_relative	0.00002	LP 目标相对改善阈值
pricing_min_lp_improvement_per_group	5.0	按 group 归一的改善阈值
pricing_min_lp_improvement_per_1000_columns	100.0	按列规模归一的改善阈值
feasibility_early_stop_enabled	False	是否启用可行性 repair 早停
feasibility_early_stop_min_iteration	1	可行性早停最小迭代轮
full_column_pool	False	是否一次性生成完整列池

7.5 ColumnGenerationConfig: 候选贝位和初始列

参数	默认值	说明
initial_columns_per_group	8	每个 group 初始列数
max_candidate_bays_per_group	500	每个 group 最多候选贝位数
adaptive_pricing_enabled	True	是否自适应扩大 pricing 候选范围

pricing_candidate_bays_initial	160	pricing 初始候选贝位数
pricing_candidate_bays_growth_per_iteration	40	每轮 pricing 候选贝位增长量
pricing_candidate_bays_high_dual	260	高 dual group 的候选贝位数
pricing_candidate_bays_unplaced	500	LP 中未放置 group 的候选贝位数
stage0_closure_enabled	True	是否在最终 repair 前补充 stage0 列
stage0_closure_max_extra_columns	1500	stage0 closure 最多额外列数

7.6 ColumnGenerationConfig: 时间预算和 SCIP

参数	默认值	说明
total_time_limit	240.0	中小计划总求解时间预算, 秒
pricing_min_lp_time_limit	12.0	每轮 LP 最小时间限制
pricing_iteration_setup_reserve	8.0	pricing 每轮预留时间
mip_time_limit	120.0	最终 MIP 时间限制
mip_gap	0.01	最终 MIP gap
verbose	True	是否打印日志
use_scip	True	是否使用 SCIP
scip_disable_symmetry	True	是否禁用 SCIP symmetry

7.7 ColumnGenerationConfig: Primal expansion 和 Repair

参数	默认值	说明
primal_expansion_columns	800	primal expansion 新增列数
max_primal_expansion_rounds	3	primal expansion 轮数
primal_expansion_reduced_cost_limit	1000000.0	primal expansion reduced cost 上限
staged_repair_reserve_fraction	0.33	最终 staged repair 预留时间比例
staged_repair_reserve_min	60.0	repair 最小预留秒数
staged_repair_reserve_max	90.0	repair 最大预留秒数
staged_repair_secondary_order_min_remaining	20.0	尝试第二 repair 顺序的最小剩余时间
staged_repair_rebuild_first_unplaced_threshold	500	未放置箱达到该值时优先 staged rebuild
repair_can_exceed_medium_plan_quota	False	repair 是否可突破中计划配额

post_repair_area_relayout_enable	True	是否启用箱区内重排
d		
post_repair_area_relayout_max_patterns	1	箱区内重排每组最多尝试 pattern 数

7.8 ColumnGenerationConfig: 需求和配额模式

参数	默认值	说明
demand_mode	original	需求模式，可选 original、medium、medium-with-doc-floor、doc-only
medium_plan_quota	None	外部中计划箱区配额，单独小计划模式使用
medium_plan_bay_quota	None	外部中计划贝位配额，单独小计划模式使用

7.9 ColumnGenerationConfig: 未放置和用户要求

参数	默认值	说明
unplaced_penalty	100000.0	未放置箱惩罚
required_area_reward	1000.0	使用用户要求箱区的奖励

7.10 ColumnGenerationConfig: 中计划集中度

参数	默认值	说明
group_area_balance_penalty	18.0	粗属性组跨箱区分布平衡惩罚
medium_concentrated_group_threshold	26	判定中计划集中组的箱量阈值
medium_small_group_area_split_penalty	2400.0	同粗属性组跨多个箱区的惩罚
medium_small_group_fragment_penalty	90.0	同粗属性组集中在较大箱区块的奖励
medium_large_group_min_area_boxes	10	中计划大组单箱区最小建议箱量
medium_large_group_small_area_penalty	900.0	大组落到过小箱区份额的惩罚

medium_large_group_area_open_penalty	0.0	打开新箱区的惩罚
medium_large_group_target_area_boxes	60	估算大组目标箱区数的单箱区目标箱量

7.11 ColumnGenerationConfig: 大计划继承

参数	默认值	说明
big_plan_area_deviation_penalty	3.0	偏离大计划箱区配额的惩罚
big_plan_fallback_tier_penalty	20.0	使用非优先大计划范围的 fallback 惩罚

7.12 ColumnGenerationConfig: E 开头出口箱区规则

参数	默认值	说明
export_e_area_max_bays_per_voyage	2	出口航次在同一 E 开头箱区最多使用的贝位起点数；设为 -1 可禁用
export_e_area_non_40_penalty	300.0	非 40/45 尺出口箱放入 E 开头箱区的软惩罚

注意：E 开头箱区并不会自动允许任何流向，仍需根据箱区功能表判断。

7.13 ColumnGenerationConfig: 小计划拆分、堆场现状和作业偏好

参数	默认值	说明
small_plan_group_area_split_penalty	80.0	细属性组跨箱区惩罚
small_plan_group_block_split_penalty	35.0	细属性组跨块惩罚
small_plan_group_bay_split_penalty	8.0	细属性组跨贝惩罚
small_plan_coarse_area_block_split_penalty	24.0	同粗属性箱区内跨块惩罚
small_plan_coarse_area_bay_split_penalty	2.5	同粗属性箱区内跨贝惩罚

existing_coarse_bay_reward	48.0	候选贝与已有堆场粗 key 匹配时的奖励
existing_coarse_neighbor_bay_reward	24.0	候选贝邻近已有匹配粗 key 时的奖励
existing_other_coarse_bay_penalty	48.0	候选贝与其他粗 key 冲突时的惩罚
existing_other_coarse_neighbor_bay_penalty	12.0	候选贝邻近其他粗 key 时的惩罚
existing_coarse_neighbor_max_bay_distance	12	计算邻近堆场状态成本的最大 bay_order 距离
berth_distance_penalty	0.02	航次泊位到箱区距离惩罚
active_loading_area_penalty	16.0	当前正在作业箱区惩罚
post_window_loading_area_reward	5.0	作业窗口之后的箱区奖励
fallback_bay_penalty	4.0	使用 fallback 贝位惩罚
non_preferred_block_penalty	6.0	使用非优先块惩罚

8 常用运行命令

8.1 跑全流程

```
cd "C:\python files\Stacking"
.\.venv\Scripts\python.exe -X utf8 -B
↪ .\flat_full_yard_plan_1.4_scip\run_flat_full_yard_plan.py --run-name test_run
↪ --large-quiet
```

8.2 单独跑中小计划

```
cd "C:\python files\Stacking"
.\.venv\Scripts\python.exe -X utf8 -B
↪ .\flat_full_yard_plan_1.4_scip\run_medium_small_column_generation.py `
--big-plan .\flat_full_yard_plan_1.4_scip\full_plan_outputs\

```

8.3 单独跑小计划

```
cd "C:\python files\Stacking"
.\.venv\Scripts\python.exe -X utf8 -B
↪ .\flat_full_yard_plan_1.4_scip\run_small_plan_from_medium.py `
```

```
--medium-plan .\flat_full_yard_plan_1.4_scip\full_plan_outputs\<<run_name>\medium_s_」  
↪ mall_plan\medium_plan.csv `
```

```
--big-plan .\flat_full_yard_plan_1.4_scip\full_plan_outputs\<<run_name>\outputs_lar_」  
↪ ge\latest_run\allocation.csv `
```

```
--run-name small_only
```